

Telecom Customer Churn Prediction

Tamanna Srivastava

2026-09-28

Installing packages

```
#install.packages("reshape")  
#install.packages("corrplot")  
#install.packages("ggplot2")  
#install.packages("dplyr")  
#install.packages("MASS")  
#install.packages("pROC")  
#install.packages("class")
```

Running Libraries

```
library(tidyverse)  
library(caret)  
library(tree)  
library(rpart)  
library(rpart.plot)  
library(maptree)  
library(party)  
library(DescTools)  
library(moments)  
library(MASS)  
library(corrplot)  
library(reshape)  
library(ggplot2)  
library(pROC)  
library(class)  
library(cluster)
```

Introduction

This report analyses customer data for BangorTelco, a telecoms provider concerned about the rate at which customers are leaving for competitors. The aim is to predict which customers are most likely to churn, so that retention spend can be targeted at the customers most at risk rather than offered to everyone. Four methods are applied: decision trees, logistic regression, k-nearest neighbours and k-means clustering.

Loading the data

```
customers <- read.csv("data/Telecom_Customers.csv")
```

```
class(customers)
```

```
## [1] "data.frame"
```

Making a copy of the dataset

```
customers_original = customers
```

Cleaning the data

Checking for duplicated records

```
customers[duplicated(customers),]
```

```
## [1] CUSTOMERID          YEAR_BIRTH
## [3] COLLEGE              INCOME
## [5] OVERAGE             LEFTOVER
## [7] HOUSE              HANDSET_PRICE
## [9] OVER_15MINS_CALLS_PER_MONTH AVERAGE_CALL_DURATION
## [11] REPORTED_SATISFACTION REPORTED_USAGE_LEVEL
## [13] CONSIDERING_CHANGE_OF_PLAN LEAVE
## <0 rows> (or 0-length row.names)
```

There are no duplicate records.

Checking for data types of variables

```
str(customers)
```

```
## 'data.frame': 20000 obs. of 14 variables:
## $ CUSTOMERID : chr "BTLC-007761" "BTLC-007682" "BTLC-002228" "BTLC-011752" ...
## $ YEAR_BIRTH : int 1975 1979 1990 1982 2002 1983 1987 1997 1974 1975 ...
## $ COLLEGE : chr "zero" "one" "zero" "one" ...
## $ INCOME : int 89318 142814 55675 39559 145081 120631 59162 117488 82304 46786 ...
## $ OVERAGE : int 0 187 0 0 0 66 0 53 170 44 ...
## $ LEFTOVER : int 0 17 32 0 0 17 55 12 34 0 ...
## $ HOUSE : int 162233 346690 792662 416439 341108 467811 251345 810740 517128 ...
## $ HANDSET_PRICE : int 266 716 257 165 583 884 396 205 369 193 ...
## $ OVER_15MINS_CALLS_PER_MONTH : int 1 24 1 0 0 4 1 4 26 5 ...
## $ AVERAGE_CALL_DURATION : int 12 4 1 15 9 6 1 4 2 8 ...
## $ REPORTED_SATISFACTION : chr "unsat" "unsat" "very_unsat" "very_sat" ...
## $ REPORTED_USAGE_LEVEL : chr "very_little" "high" "very_little" "high" ...
## $ CONSIDERING_CHANGE_OF_PLAN : chr "considering" "considering" "never_thought" "considering" ...
## $ LEAVE : chr "STAY" "LEAVE" "STAY" "STAY" ...
```

```
# converting all character variables to factors for better analysis
customers[,c(1,3,11,12,13,14)] = lapply(customers[,c(1,3,11,12,13,14)],factor)
```

checking for missing values using map function from purrr library.

```
map(customers,~sum(is.na(.)))
```

```
## $CUSTOMERID
## [1] 0
##
## $YEAR_BIRTH
## [1] 0
##
## $COLLEGE
## [1] 0
##
## $INCOME
## [1] 0
##
## $OVERAGE
## [1] 0
##
## $LEFTOVER
## [1] 0
##
## $HOUSE
## [1] 0
##
## $HANDSET_PRICE
## [1] 0
##
## $OVER_15MINS_CALLS_PER_MONTH
## [1] 0
##
## $AVERAGE_CALL_DURATION
## [1] 0
##
## $REPORTED_SATISFACTION
## [1] 0
##
## $REPORTED_USAGE_LEVEL
## [1] 0
##
## $CONSIDERING_CHANGE_OF_PLAN
## [1] 0
##
## $LEAVE
## [1] 0
```

There are no missing values.

checking for errors

```
# considering input errors in Year_Birth column
error <- customers[customers$YEAR_BIRTH<1924,"YEAR_BIRTH"]
error
```

```
## [1] 1907 1915 1911
```

```
neg_overage <- customers[customers$OVERAGE<0,]
neg_overage
```

```
##      CUSTOMERID YEAR_BIRTH COLLEGE INCOME OVERAGE LEFTOVER  HOUSE
## 19034 BTLC-014633      1977    zero  92956      -2         0 816777
##      HANDSET_PRICE OVER_15MINS_CALLS_PER_MONTH AVERAGE_CALL_DURATION
## 19034          274                        4                        2
##      REPORTED_SATISFACTION REPORTED_USAGE_LEVEL CONSIDERING_CHANGE_OF_PLAN
## 19034          very_unsat          very_high          considering
##      LEAVE
## 19034  STAY
```

```
# removing these records
customers <- customers[!customers$YEAR_BIRTH<1924,]
customers <- customers[!customers$OVERAGE<0,]
```

checking for unique customer IDs

```
length(unique(customers$CUSTOMERID))
```

```
## [1] 19996
```

There are 19996 unique customer IDs after removing the rows having errors.

Now making a copy of the cleaned dataset and removing the customer ID column for easier application of ML models

```
customers_copy <- customers
customers <- customers[-1]
```

checking for normality

```
z <- customers %>%
  summarise(skewness(INCOME), kurtosis(INCOME), skewness(OVERAGE), kurtosis(OVERAGE), skewness(LEFTOVER), ku
z
```

```
## skewness(INCOME) kurtosis(INCOME) skewness(OVERAGE) kurtosis(OVERAGE)
## 1 0.3023425 1.80958 0.5654534 1.791695
## skewness(LEFTOVER) kurtosis(LEFTOVER) skewness(HOUSE) kurtosis(HOUSE)
## 1 0.9822683 2.678844 0.4104356 1.88875
## skewness(HANDSET_PRICE) kurtosis(HANDSET_PRICE)
## 1 0.8477806 2.53906
## skewness(OVER_15MINS_CALLS_PER_MONTH) kurtosis(OVER_15MINS_CALLS_PER_MONTH)
## 1 1.015548 2.591675
## skewness(AVERAGE_CALL_DURATION) kurtosis(AVERAGE_CALL_DURATION)
## 1 0.6257171 2.135691
```

All variables have skewness between -1 and 1 (OVER_15MINS_CALLS_PER_MONTH marginally above 1), so none is strongly skewed. Kurtosis values sit between 1.8 and 2.7, below the value of 3 expected under a normal distribution, so the distributions are flatter than normal. None of the methods used in this report assumes normally distributed predictors, so this check is descriptive rather than a test of a model assumption.

Making a new transformed variable for ‘OVER_15MINS_CALLS_PER_MONTH’

This can be used later in the analysis, if required, for improving accuracy of ML models. Square root transformation has been applied on this variable. Square root transformation technique is an effective method for normality correction (Ozdemir, 2016).

```
customers$OVER_15MINS_CALLS_PER_MONTH_transformed <- sqrt(customers$OVER_15MINS_CALLS_PER_MONTH)
# rechecking the skewness kurtosis values for normality
skewness(customers$OVER_15MINS_CALLS_PER_MONTH_transformed)
```

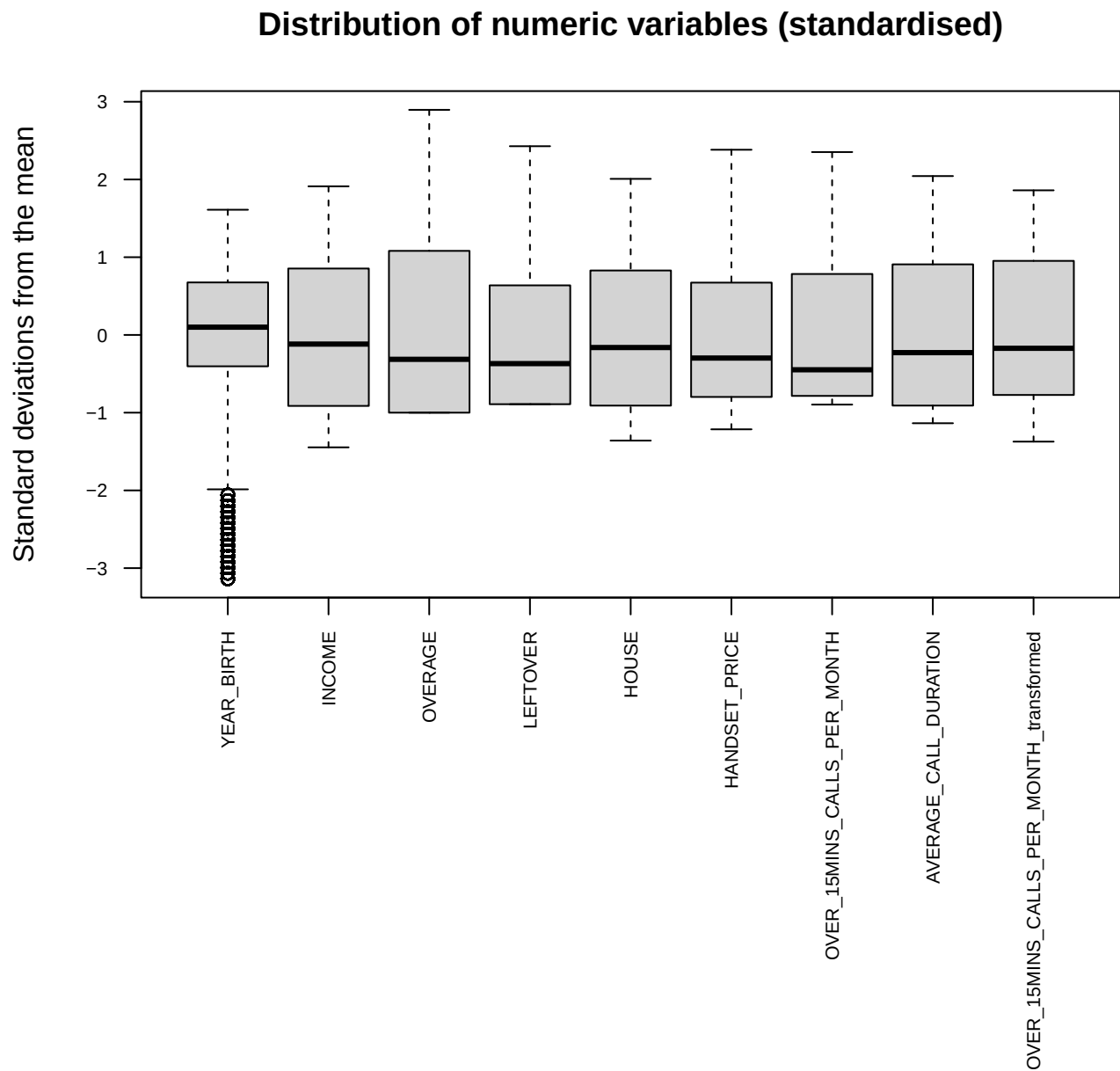
```
## [1] 0.3397497
```

```
kurtosis(customers$OVER_15MINS_CALLS_PER_MONTH_transformed)
```

```
## [1] 1.949078
```

Checking the distribution of numeric variables for outliers

```
customers_subset <- customers[, sapply(customers, is.numeric)]
par(mar = c(16, 4, 4, 2))
boxplot(scale(customers_subset), las = 2, cex.axis = 0.7,
        main = "Distribution of numeric variables (standardised)",
        ylab = "Standard deviations from the mean")
```



Once standardised, all variables sit within a comparable range. The points below YEAR_BIRTH are the oldest customers in the dataset, flagged by the boxplot's 1.5 IQR rule because birth years are tightly clustered. These are valid records rather than data errors, since the implausible birth years were removed during cleaning.

Making four different train and test samples

```

set.seed(100)
train_index <- sample(seq_len(nrow(customers)), size = nrow(customers)*0.7,replace=FALSE)
TrainData <- customers[train_index,]
TestData <- customers[-train_index,]

set.seed(100)
train_index2 <- sample(seq_len(nrow(customers)), size = nrow(customers)*0.8,replace=FALSE)
TrainData2 <- customers[train_index2,]
TestData2 <- customers[-train_index2,]

set.seed(100)
train_index3 <- createDataPartition(y=customers$LEAVE,p=0.7,list=FALSE)
TrainData3 <- customers[train_index3,]
TestData3 <- customers[-train_index3,]

set.seed(100)
train_index4 <- createDataPartition(y=customers$LEAVE,p=0.8,list=FALSE)
TrainData4 <- customers[train_index4,]
TestData4 <- customers[-train_index4,]

```

TASK 1 - The best decision tree model

Decision-Tree Model with TrainData4 (80:20 ratio) create data partition and rpart function

Eight decision-tree models were built for this task, varying the train/test ratio (70:30 and 80:20), the sampling method (simple random sampling and stratified sampling using createDataPartition), and the package and control parameters (tree and rpart). All eight are reported in the appendix, with out-of-sample accuracies ranging from 66.89% to 71.23%. They were fitted on four different test sets, so these figures are not directly comparable, and much of the spread reflects variation between splits rather than genuine differences between the models. Cross-validation over identical folds would give a cleaner comparison.

The model below was selected. It uses a stratified 80:20 split with rpart defaults and gives an out-of-sample accuracy of 68.68% against an in-sample accuracy of 69.45%. The small gap between the two indicates the tree is not overfitting. It was chosen because it is a readable seven-node tree at comparable accuracy. The highest scoring model produces fourteen terminal nodes and a plot that cannot be read, which does not give a handful of clear rules that the sales team can use to target retention effort.

```

set.seed(100)
Tree8 <- rpart(LEAVE ~ ., data = TrainData4, method = "class")
Tree8Pred <- predict(Tree8, TrainData4, type = "class")
confusionMatrix(Tree8Pred, TrainData4$LEAVE)

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction LEAVE STAY
##      LEAVE   6520 3527
##      STAY    1360 4591
##
##              Accuracy : 0.6945
##              95% CI : (0.6873, 0.7017)

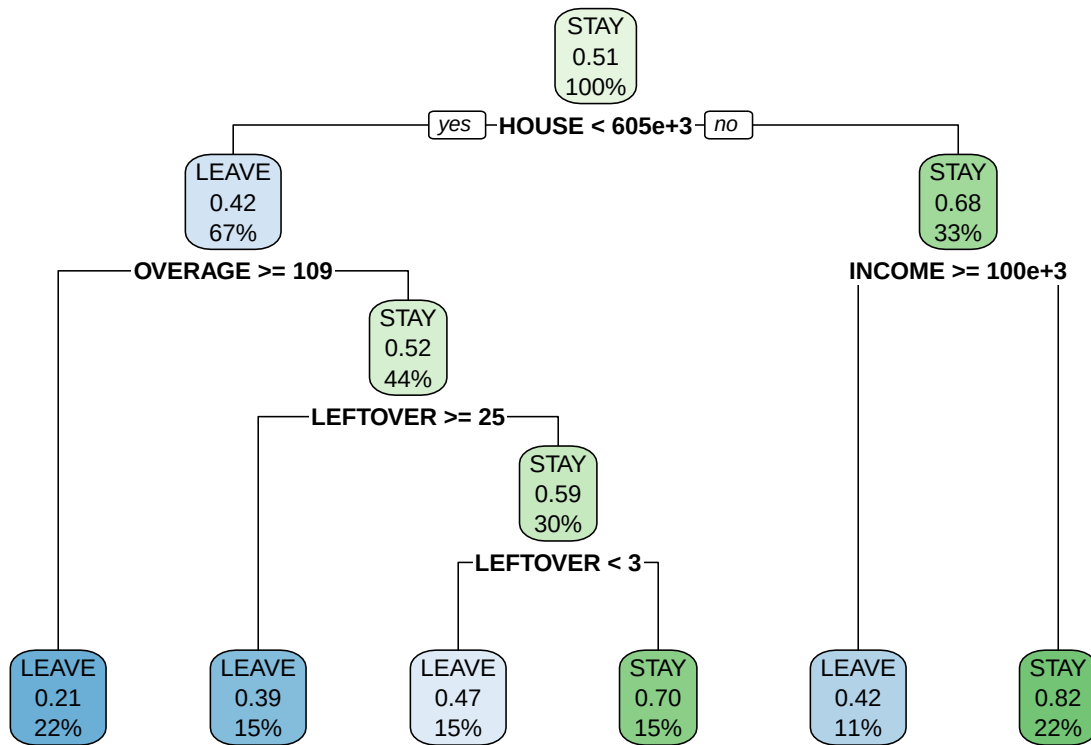
```

```
##      No Information Rate : 0.5074
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.3914
##
##      McNemar's Test P-Value : < 2.2e-16
##
##              Sensitivity : 0.8274
##              Specificity : 0.5655
##              Pos Pred Value : 0.6489
##              Neg Pred Value : 0.7715
##              Prevalence : 0.4926
##              Detection Rate : 0.4076
##      Detection Prevalence : 0.6280
##      Balanced Accuracy : 0.6965
##
##      'Positive' Class : LEAVE
##
```

```
Tree8Pred <- predict(Tree8, TestData4, type = "class")
confusionMatrix(Tree8Pred, TestData4$LEAVE)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction LEAVE STAY
##      LEAVE  1616  899
##      STAY   353 1130
##
##              Accuracy : 0.6868
##              95% CI : (0.6722, 0.7012)
##      No Information Rate : 0.5075
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.3761
##
##      McNemar's Test P-Value : < 2.2e-16
##
##              Sensitivity : 0.8207
##              Specificity : 0.5569
##              Pos Pred Value : 0.6425
##              Neg Pred Value : 0.7620
##              Prevalence : 0.4925
##              Detection Rate : 0.4042
##      Detection Prevalence : 0.6291
##      Balanced Accuracy : 0.6888
##
##      'Positive' Class : LEAVE
##
```

```
rpart.plot(Tree8)
```



Interpretation of the decision tree

In each node of the chart, `rpart.plot` shows the predicted class, the probability of staying (STAY is the second factor level), and the percentage of all customers falling into that node. At the root, 51% of customers stay overall.

The first split is on house value. Customers with a house value below £605,000 (67% of all customers) have a 42% probability of staying, so around 58% of this group leave. Customers with a house value of £605,000 or more (33% of customers) have a 68% probability of staying.

Within the lower house-value branch, the next split is on overage. Customers overcharged £109 or more per month (22% of all customers) have only a 21% chance of staying, making them the highest-risk segment in the tree. Customers overcharged less than £109 (44% of customers) have a 52% chance of staying and split further on leftover minutes. Those with 25% or more of their minutes left over each month (15% of customers) have a 39% chance of staying. Those with less than 25% leftover (30% of customers) have a 59% chance of staying and split once more: less than 3% leftover (15% of customers) gives a 47% chance of staying, while between 3% and 25% leftover (15% of customers) gives the strongest retention in this branch at 70%.

Within the higher house-value branch, the split is on income. Perhaps counterintuitively, customers earning £100,000 or more (11% of all customers) have only a 42% chance of staying, while those earning less than £100,000 (22% of customers) form the most loyal segment in the tree, with an 82% chance of staying.

Recommendations to management

The retention budget should be directed at the highest-risk segments the tree identifies. The clearest signal is heavy overage: customers with house values below £605,000 who are overcharged £109 or more per month are around 79% likely to leave, so reviewing their plans or capping overage charges should be the first priority. The second at-risk group is higher house-value customers earning £100,000 or more, who are around 58% likely to leave and may need a different retention approach, such as premium plan benefits. Customers with a large share of leftover minutes (61% likely to leave) appear to be paying for plans larger than they need, and moving them to cheaper plans may retain them. The most loyal segment, house value above £605,000 with income below £100,000 (82% likely to stay), requires little immediate retention spend.

Task 2 - The best Logistic Regression Model

```
set.seed(100)
train_index2 <- sample(seq_len(nrow(customers_copy)), size = nrow(customers_copy)*0.8,replace=FALSE)
TrainData2 <- customers_copy[train_index2,]
TestData2 <- customers_copy[-train_index2,]

set.seed(100)
logistics_1<-train(data=TrainData2,LEAVE~INCOME+OVERAGE+LEFTOVER+HOUSE+HANDSET_PRICE+AVERAGE_CALL_DURATION)

summary(logistics_1)

##
## Call:
## NULL
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    4.442e-01  6.994e-02   6.352 2.12e-10 ***
## INCOME         -3.928e-06  5.888e-07  -6.671 2.53e-11 ***
## OVERAGE        -6.037e-03  2.013e-04 -29.994 < 2e-16 ***
## LEFTOVER       -7.867e-03  8.394e-04  -9.372 < 2e-16 ***
## HOUSE          1.836e-06  6.837e-08  26.849 < 2e-16 ***
## HANDSET_PRICE  -3.150e-04  1.150e-04  -2.740 0.00615 **
## AVERAGE_CALL_DURATION -2.705e-02  5.118e-03  -5.285 1.26e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 22168  on 15995  degrees of freedom
## Residual deviance: 20271  on 15989  degrees of freedom
## AIC: 20285
##
## Number of Fisher Scoring iterations: 4

pred1<-predict(logistics_1,newdata=TestData2)

confusionMatrix(pred1,TestData2$LEAVE)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction LEAVE STAY
##      LEAVE 1234  625
##      STAY  781 1360
##
##           Accuracy : 0.6485
##           95% CI : (0.6335, 0.6633)
##      No Information Rate : 0.5038
##      P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.2974
##
## Mcnemar's Test P-Value : 3.57e-05
##
##           Sensitivity : 0.6124
##           Specificity : 0.6851
##      Pos Pred Value : 0.6638
##      Neg Pred Value : 0.6352
##           Prevalence : 0.5038
##      Detection Rate : 0.3085
##      Detection Prevalence : 0.4647
##      Balanced Accuracy : 0.6488
##
##      'Positive' Class : LEAVE
##
```

```
pred1_prob <- predict(logistics_1,newdata=TestData2,type = 'prob')
head(pred1_prob)
```

```
##      LEAVE      STAY
## 6  0.5358703 0.4641297
## 12 0.4697577 0.5302423
## 21 0.4155313 0.5844687
## 39 0.3771365 0.6228635
## 40 0.5270354 0.4729646
## 45 0.7529408 0.2470592
```

This is the best logistic regression model, with an out-of-sample accuracy of 64.85%. The sensitivity is 61.24%, meaning the model correctly identified 61.24% of the customers who actually left (true positives). The specificity is 68.51%, meaning it correctly identified 68.51% of the customers who actually stayed.

```
# Getting predicted probabilities for only the LEAVE class from the dataframe of predicted probabilities
```

```
pred1_prob_leave <- pred1_prob[, "LEAVE"]
```

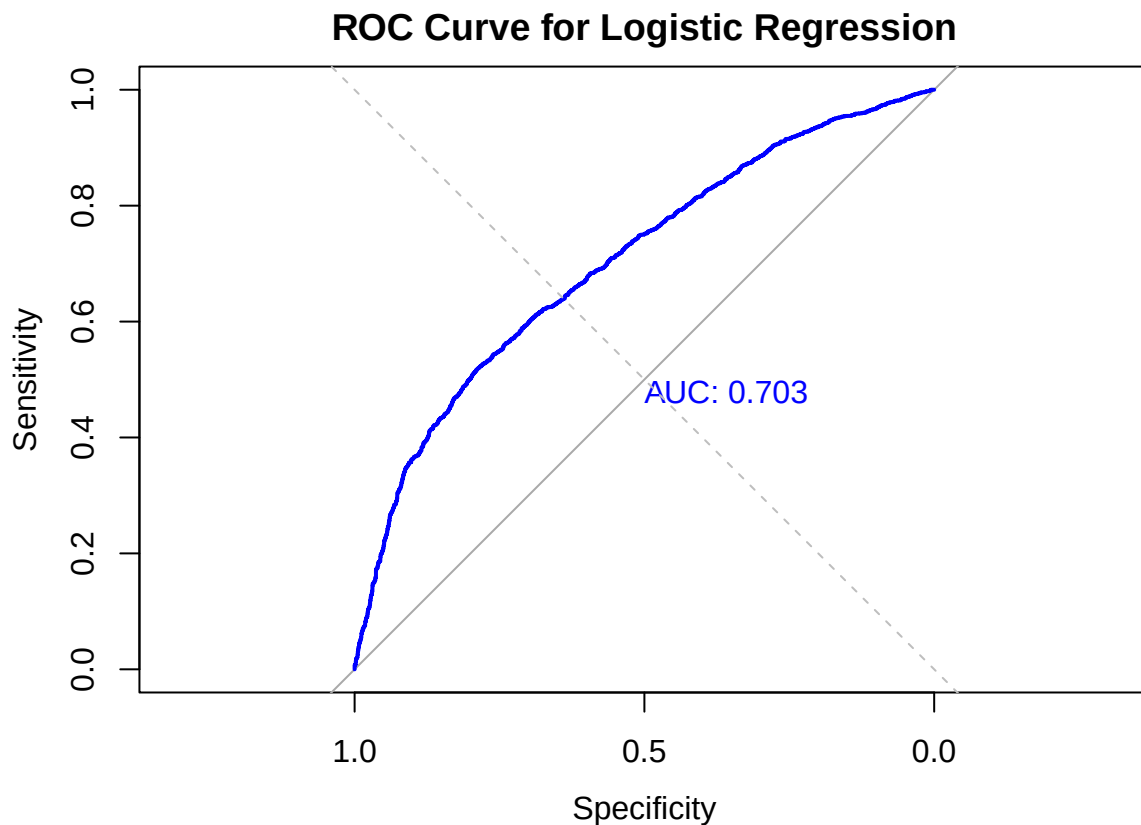
```
# Creating and plot ROC curve
```

```
roc_obj <- roc(TestData2$LEAVE, pred1_prob_leave,
               levels = c("STAY", "LEAVE"), direction = "<")
```

```
# Basic plot
```

```
plot(roc_obj,
     main = "ROC Curve for Logistic Regression",
     col = "blue",
     lwd = 2,
     print.auc = TRUE)
```

```
# Adding a diagonal reference line
abline(a = 0, b = 1, lty = 2, col = "gray")
```

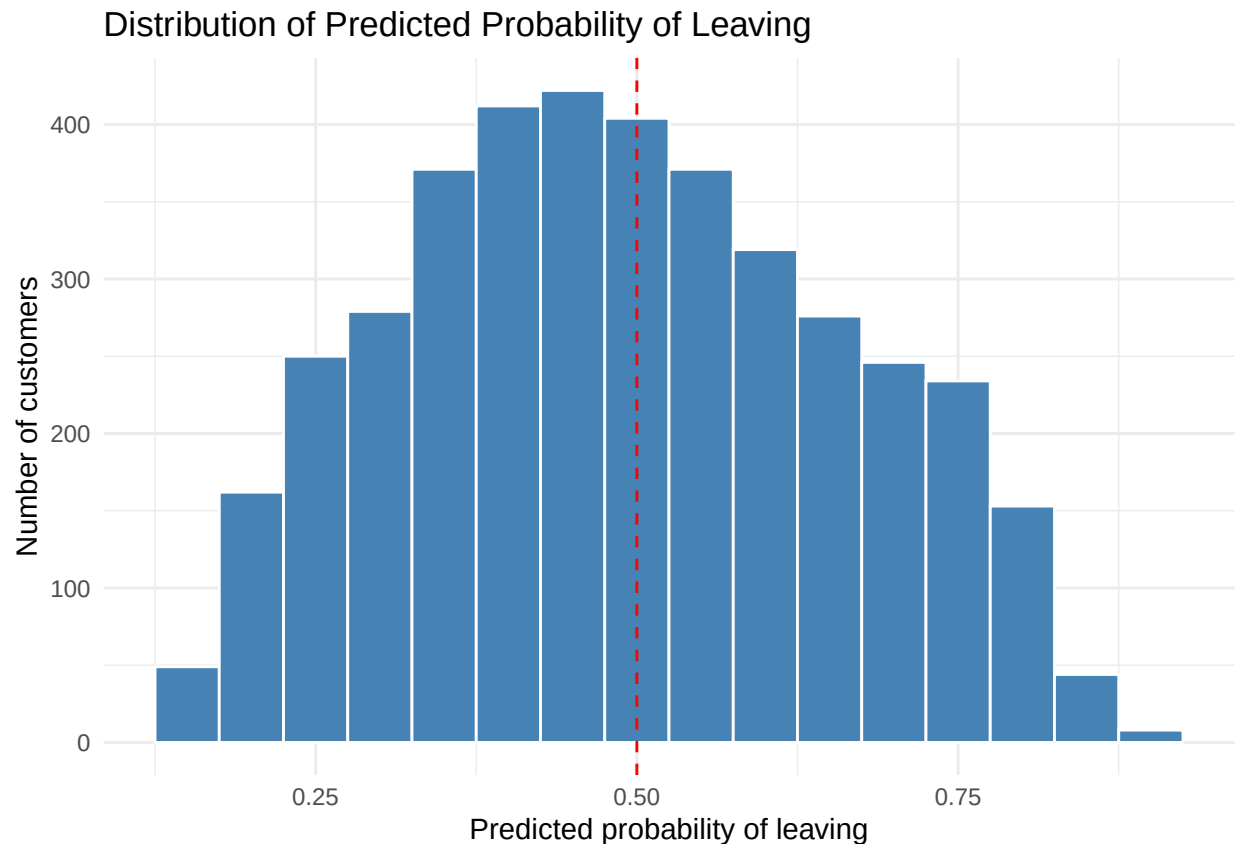


Interpretation of the ROC curve

The ROC curve for this logistic regression model provides insights into how effectively and accurately the model distinguishes between customers who are likely to leave and those who are likely to stay. The curve represents sensitivity (true positive rate) against 1-specificity (false positive rate), offering a visual representation of the model's classification performance. The Area Under the Curve (AUC) is **0.703**, which suggests that the model has a **moderate ability to discriminate** between the two groups. According to Dreiseitl and Ohno-Machado (2002), an AUC of 0.5 indicates a model with no predictive power, while an AUC of 1.0 represents a perfect classifier. An AUC of 0.703 means that if one customer who left and one who stayed are selected at random, the model assigns a higher predicted probability of leaving to the actual leaver about 70.3% of the time.

```
# Distribution of predicted probabilities of leaving across the test set
ggplot(pred1_prob, aes(x = LEAVE)) +
  geom_histogram(binwidth = 0.05, fill = "steelblue", colour = "white") +
```

```
geom_vline(xintercept = 0.5, linetype = "dashed", colour = "red") +
labs(title = "Distribution of Predicted Probability of Leaving",
     x = "Predicted probability of leaving",
     y = "Number of customers") +
theme_minimal()
```



The histogram shows the distribution of predicted probabilities of leaving assigned to each customer from the logistic regression model. Customers to the right of the dashed line at 0.5 are classified as likely to leave, those to the left as likely to stay.

The distribution is roughly symmetric with a single peak just below 0.5, and slightly more customers fall below the threshold than above it. Almost no customers receive probabilities close to 0 or 1 - the model assigns mid-range probabilities to most of the test set.

This is consistent with the moderate AUC of 0.703 and indicates that a large number of customers sit near the 0.5 boundary where classifications are least certain and most sensitive to the choice of threshold.

TASK 3 - KNN MODEL

```
# making train and test dataset
set.seed(100)
train_index3 <- createDataPartition(y=customers_copy$LEAVE,p=0.7,list=FALSE)
TrainData3 <- customers_copy[train_index3,]
TestData3 <- customers_copy[-train_index3,]
```

```

# dealing with year column since it is numeric and needs to be scaled for KNN but scaling the Years col

# current year hardcoded to 2026 for results to stay reproducible
current_year <- 2026

# Converting birth year to age
TrainData3$AGE <- current_year - TrainData3$YEAR_BIRTH
TestData3$AGE <- current_year - TestData3$YEAR_BIRTH

# Dropping the original birth year column
TrainData3$YEAR_BIRTH <- NULL
TestData3$YEAR_BIRTH <- NULL

# scaling the numerical columns using training-set means and SDs to avoid data leakage
numeric_cols <- sapply(TrainData3, is.numeric)
train_means <- sapply(TrainData3[, numeric_cols], mean)
train_sds <- sapply(TrainData3[, numeric_cols], sd)
TrainData3[, numeric_cols] <- scale(TrainData3[, numeric_cols])
TestData3[, numeric_cols] <- scale(TestData3[, numeric_cols], center = train_means, scale = train_sds)

# finding out the best k
# Cross-validation to know best k
set.seed(100)
control <- trainControl(method = "cv", number = 10)
knn_cv <- train(
  LEAVE ~ .,
  data = TrainData3[, -c(1,2,10,11,12)],
  method = "knn",
  tuneGrid = data.frame(k = 1:50),
  trControl = control
)

# Best k value
best_k <- knn_cv$bestTune$k
print(best_k)

```

```
## [1] 41
```

The full cross-validation results table contains one row for each of the fifty values of k tested. It is not printed here for length; the selected value is shown above.

```
print(knn_cv$results)
```

```

# making the KNN model
set.seed(100)
knn_predictions <- knn(
  train = TrainData3[, -c(1,2,10,11,12,13)],
  test = TestData3[, -c(1,2,10,11,12,13)],
  cl = TrainData3$LEAVE,
  k = 41
)

confusionMatrix(knn_predictions, TestData3$LEAVE)

```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction LEAVE STAY
##      LEAVE  2043  992
##      STAY   911 2052
##
##           Accuracy : 0.6827
##           95% CI : (0.6708, 0.6945)
##      No Information Rate : 0.5075
##      P-Value [Acc > NIR] : < 2e-16
##
##           Kappa : 0.3656
##
## Mcnemar's Test P-Value : 0.06667
##
##           Sensitivity : 0.6916
##           Specificity : 0.6741
##      Pos Pred Value : 0.6731
##      Neg Pred Value : 0.6925
##           Prevalence : 0.4925
##      Detection Rate : 0.3406
##      Detection Prevalence : 0.5060
##      Balanced Accuracy : 0.6829
##
##           'Positive' Class : LEAVE
##
```

The K-Nearest Neighbours (KNN) model achieved an out-of-sample accuracy of **68.27%**, meaning it correctly predicted customer behaviour in 68.27% of test cases. The model's **sensitivity is 69.16%**, indicating that it correctly identifies 69.16% of the customers who actually leave. Its specificity of 67.41% means it correctly identifies **67.41%** of those who stay. Performance is therefore well balanced across both classes. The model beats the logistic regression on accuracy (68.27% against 64.85%) and on sensitivity (69.16% against 61.24%), while the logistic regression is marginally better on specificity (68.51% against 67.41%). The sensitivity difference is the one that matters here, since the business objective is identifying customers at risk of leaving.

The value of **k = 41** was selected using **10-fold cross-validation**, with the `trainControl` function in R (The R Documentation, n.d.). The training dataset is split into ten parts, with nine used for training and one for validation in each iteration, and the value of k with the best average validation performance is chosen. This gives a more reliable basis for selecting k than a single split would.

Only numeric variables were used, since KNN relies on Euclidean distance and the categorical fields (`COLLEGE`, `REPORTED_SATISFACTION`, `REPORTED_USAGE_LEVEL` and `CONSIDERING_CHANGE_OF_PLAN`) have no meaningful numeric spacing. `CUSTOMERID` was excluded as an identifier with no predictive content, and `YEAR_BIRTH` was converted to `AGE`. All numeric predictors were standardised using the training set's means and standard deviations, so that variables measured on large scales such as `INCOME` and `HOUSE` do not dominate the distance calculation.

```
# Performing PCA on the data
pca_knn_train <- prcomp(TrainData3[, -c(1,2,10,11,12,13)])
pca_knn_train
```

```
## Standard deviations (1, ..., p=8):
```

```
## [1] 1.3299419 1.3156755 1.2870713 1.0014254 0.9983905 0.5813633 0.5217365
## [8] 0.4836025
##
## Rotation (n x k) = (8 x 8):
##
##          PC1          PC2          PC3          PC4
## INCOME      0.122516947  0.675746454 -0.167844097  0.001982570
## OVERAGE     -0.689594214  0.141768910  0.064721902  0.011983859
## LEFTOVER      0.093519224  0.153986963  0.683754721  0.004817970
## HOUSE       -0.019287553 -0.009294643  0.012456067 -0.676392113
## HANDSET_PRICE 0.122423784  0.674687817 -0.172494866 -0.012529903
## OVER_15MINS_CALLS_PER_MONTH -0.690220588  0.140519337  0.060776519  0.008653277
## AVERAGE_CALL_DURATION -0.094541149 -0.156460119 -0.682967793 -0.011979551
## AGE          0.001220705 -0.006389662 -0.008395995  0.736170881
##
##          PC5          PC6          PC7
## INCOME      0.016302887  0.002827146 -0.7070189042
## OVERAGE     -0.009576810  0.005734356 -0.0051183325
## LEFTOVER    -0.000170429 -0.707095311 -0.0017843384
## HOUSE        0.736112629  0.002699856 -0.0001212623
## HANDSET_PRICE 0.003244945 -0.005524661  0.7069850249
## OVER_15MINS_CALLS_PER_MONTH -0.006174697 -0.005252913  0.0057213020
## AVERAGE_CALL_DURATION -0.001345632 -0.707028583 -0.0067394668
## AGE          0.676557612 -0.004530271  0.0137438720
##
##          PC8
## INCOME     -5.590036e-03
## OVERAGE      7.070206e-01
## LEFTOVER     3.383323e-03
## HOUSE        3.324122e-03
## HANDSET_PRICE 5.330355e-03
## OVER_15MINS_CALLS_PER_MONTH -7.070946e-01
## AVERAGE_CALL_DURATION  7.552339e-03
## AGE          6.289985e-05
```

Principal Component Analysis (PCA) Interpretation

The PCA results help to understand how the original features contribute to the newly created principal components (PC1 and PC2). Looking at the rotation matrix, PC1 is primarily influenced by the features `OVER_15MINS_CALLS_PER_MONTH` (-0.6902) and `OVERAGE` (-0.6896), indicating that these factors have the strongest impact on this component. The negative values suggest that as these feature values increase, PC1 decreases. On the other hand, PC2 is heavily influenced by `HANDSET_PRICE` (0.6747) and `INCOME` (0.6757), showing that these variables play a major role in differentiating customers based on PC2.

```
summary(pca_knn_train)
```

```
## Importance of components:
##          PC1    PC2    PC3    PC4    PC5    PC6    PC7
## Standard deviation  1.3299 1.3157 1.2871 1.0014 0.9984 0.58136 0.52174
## Proportion of Variance 0.2211 0.2164 0.2071 0.1254 0.1246 0.04225 0.03403
## Cumulative Proportion 0.2211 0.4375 0.6445 0.7699 0.8945 0.93674 0.97077
##
##          PC8
## Standard deviation  0.48360
## Proportion of Variance 0.02923
## Cumulative Proportion 1.00000
```

The importance of components table reveals that PC1 and PC2 together account for 43.75% of the total variance, meaning they capture a significant portion of the dataset's overall variability, while the remaining variance is spread across other components.

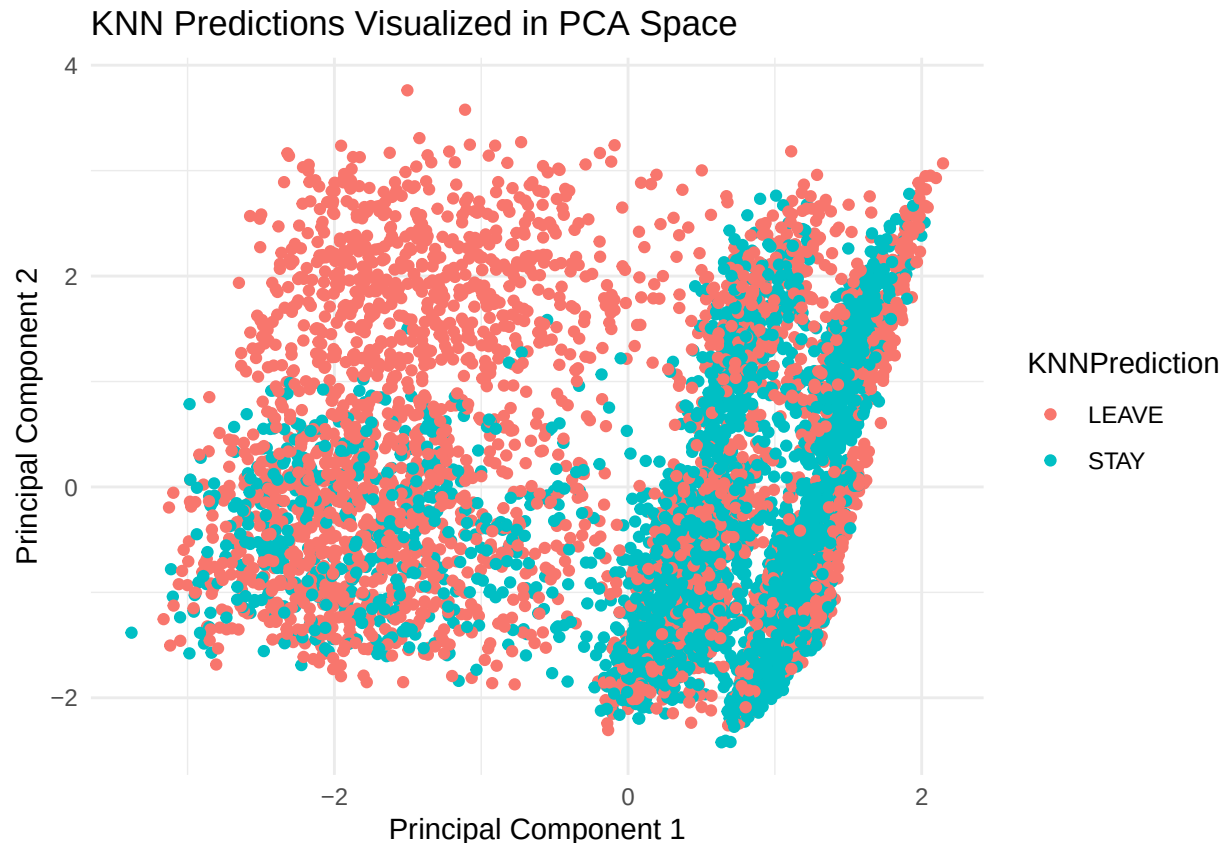
```
# TestData3 is already scaled with training-set statistics above;  
# predict() applies the PCA centering itself, so no further scaling is needed here  
pca_train_data <- data.frame(PCA1 = pca_knn_train$x[, 1], PCA2 = pca_knn_train$x[, 2])  
  
pca_test_data <- predict(pca_knn_train, TestData3[, -c(1,2,10,11,12,13)] )  
  
pca_test_data <- data.frame(PCA1 = pca_test_data[, 1], PCA2 = pca_test_data[, 2])  
  
# Ensuring knn_predictions length matches test data  
pca_test_data$KNNPrediction <- knn_predictions  
  
# Checking the structure and dimensions  
print(dim(pca_test_data))
```

```
## [1] 5998    3
```

```
length(knn_predictions)
```

```
## [1] 5998
```

```
# Visualising  
ggplot(pca_test_data, aes(x = PCA1, y = PCA2, color = KNNPrediction)) +  
  geom_point() +  
  labs(title = "KNN Predictions Visualized in PCA Space",  
        x = "Principal Component 1",  
        y = "Principal Component 2") +  
  theme_minimal()
```



Interpreting the scatter plot

The scatter plot shows the KNN predictions for the test dataset projected onto the first two principal components. Each point represents a customer, coloured by the class the model predicted, **LEAVE** or **STAY**. There is a noticeable pattern where customers predicted to stay cluster on the right side of the chart, while those predicted to leave are more concentrated on the left. This separation is meaningful: **PC1** is driven by **OVERAGE** and **OVER_15MINS_CALLS_PER_MONTH**, so the right-hand side represents low-overage customers. This agrees with the decision tree, which identified high overage as the strongest churn signal, and the two methods therefore point to the same driver.

The groups overlap in the middle, but that is expected in a two-dimensional view of an eight-variable model. The model's performance is the test set result above, at 68.27%.

TASK 4 - Clustering

```
set.seed(100)
train_index4 <- createDataPartition(y=customers_copy$LEAVE,p=0.8,list=FALSE)
TrainData4 <- customers_copy[train_index4,]
TestData4 <- customers_copy[-train_index4,]

# Converting birth year to age
# Age is hardcoded to 2026 for results to stay reproducible if this file is knitted in a later year
current_year <- 2026
```

```

TrainData4$AGE <- current_year - TrainData4$YEAR_BIRTH
TestData4$AGE <- current_year - TestData4$YEAR_BIRTH

# Drop the original birth year column
TrainData4$YEAR_BIRTH <- NULL
TestData4$YEAR_BIRTH <- NULL

# making copies of the datasets before scaling
TrainData4_original <- TrainData4
TestData4_original <- TestData4

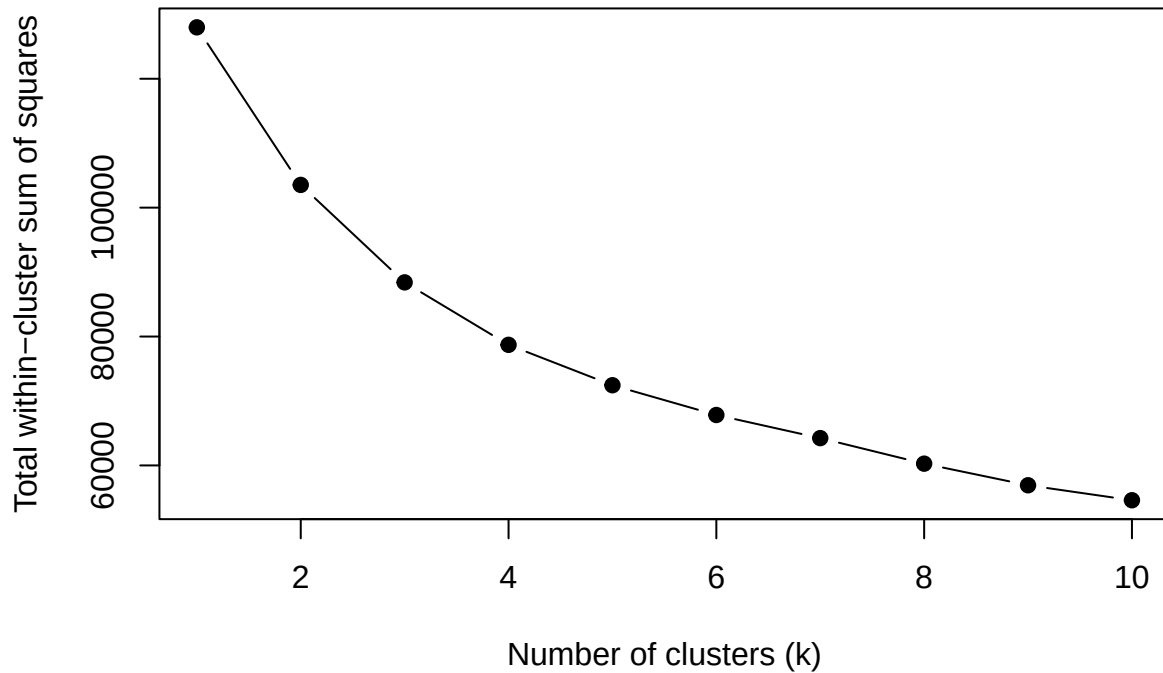
# scaling the numerical columns using training-set means and SDs to avoid data leakage
numeric_cols <- sapply(TrainData4, is.numeric)
train_means4 <- sapply(TrainData4[, numeric_cols], mean)
train_sds4 <- sapply(TrainData4[, numeric_cols], sd)
TrainData4[, numeric_cols] <- scale(TrainData4[, numeric_cols])
TestData4[, numeric_cols] <- scale(TestData4[, numeric_cols], center = train_means4, scale = train_sds4)

# Choosing the number of clusters using the elbow method
set.seed(100)
wss <- sapply(1:10, function(k) {
  kmeans(TrainData4[, -c(1,2,10,11,12,13)], centers = k, nstart = 25)$tot.withinss
})

plot(1:10, wss, type = "b", pch = 19,
     xlab = "Number of clusters (k)",
     ylab = "Total within-cluster sum of squares",
     main = "Elbow Method for Choosing k")

```

Elbow Method for Choosing k



```
set.seed(100)
kmeans_cluster_model1 <- kmeans(TrainData4[, -c(1,2,10,11,12,13)], centers = 3, nstart = 25)

pca <- prcomp(TrainData4[, -c(1,2,10,11,12,13)])
pca
```

```
## Standard deviations (1, ..., p=8):
## [1] 1.3314843 1.3140308 1.2884318 1.0000621 0.9996037 0.5823329 0.5231472
## [8] 0.4777971
##
## Rotation (n x k) = (8 x 8):
##
```

	PC1	PC2	PC3	PC4
## INCOME	-0.05788256	0.70192728	0.060933177	0.010246909
## OVERAGE	0.70219685	0.06120575	-0.054932119	0.013550360
## LEFTOVER	-0.05856798	0.05628788	-0.702261212	0.010374406
## HOUSE	0.01100351	-0.01170856	-0.024800289	-0.507837782
## HANDSET_PRICE	-0.05641029	0.70193196	0.063214483	-0.006839722
## OVER_15MINS_CALLS_PER_MONTH	0.70244523	0.06251488	-0.050433168	0.009626286
## AVERAGE_CALL_DURATION	0.05738949	-0.05885515	0.702079691	-0.012605348
## AGE	-0.01062732	-0.01288498	0.005317703	0.861049479

```
##
```

	PC5	PC6	PC7
## INCOME	0.0160025306	0.0035235887	0.706869947
## OVERAGE	-0.0066264554	-0.0032812913	-0.012582514
## LEFTOVER	-0.0091184482	-0.7071101220	0.003521090
## HOUSE	0.8609110621	0.0042055637	0.002397117

```
## HANDSET_PRICE          0.0101364830 -0.0059019771 -0.706951078
## OVER_15MINS_CALLS_PER_MONTH 0.0009967521  0.0018481282  0.013629443
## AVERAGE_CALL_DURATION  0.0146934627 -0.7070472455  0.006000279
## AGE                    0.5080643331  0.0005913868 -0.012522894
##                               PC8
## INCOME                  -0.0140084764
## OVERAGE                  -0.7069365564
## LEFTOVER                 0.0047706048
## HOUSE                   -0.0060228911
## HANDSET_PRICE           0.0122123843
## OVER_15MINS_CALLS_PER_MONTH 0.7069909858
## AVERAGE_CALL_DURATION   0.0001500498
## AGE                     -0.0001226785
```

The PCA for the clustering model is computed on the eight standardised numeric variables and describes where the variation in the data lies, independently of the clusters themselves. The first two components have standard deviations of 1.33 and 1.31.

PC1 is dominated by OVERAGE (0.702) and OVER_15MINS_CALLS_PER_MONTH (0.702), which together form a heavy-usage dimension, since customers who exceed their allowance also make more long calls. A high PC1 score therefore indicates high usage and overage. PC2 is dominated by INCOME (0.702) and HANDSET_PRICE (0.702), representing customer affluence, since higher earners tend to buy more expensive handsets. PC3 contrasts AVERAGE_CALL_DURATION (0.702) with LEFTOVER (-0.702), separating customers who make longer calls from those with unused minutes.

HOUSE and AGE load close to zero on all three of these components and appear only in PC4 and PC5, indicating that they vary independently of the usage and spending variables.

```
# pca table of importance
summary(pca)
```

```
## Importance of components:
##               PC1    PC2    PC3    PC4    PC5    PC6    PC7
## Standard deviation  1.3315 1.3140 1.2884 1.000 0.9996 0.58233 0.52315
## Proportion of Variance 0.2216 0.2158 0.2075 0.125 0.1249 0.04239 0.03421
## Cumulative Proportion 0.2216 0.4374 0.6450 0.770 0.8949 0.93725 0.97146
##               PC8
## Standard deviation   0.47780
## Proportion of Variance 0.02854
## Cumulative Proportion 1.00000
```

According to the table of importance results, the first three principal components (PC1, PC2, and PC3) collectively account for almost 64.50% of the variance in the data; their respective contributions are 22.16%, 21.58%, and 20.75%. This implies that most of the basic trends in the dataset are captured by these three elements.

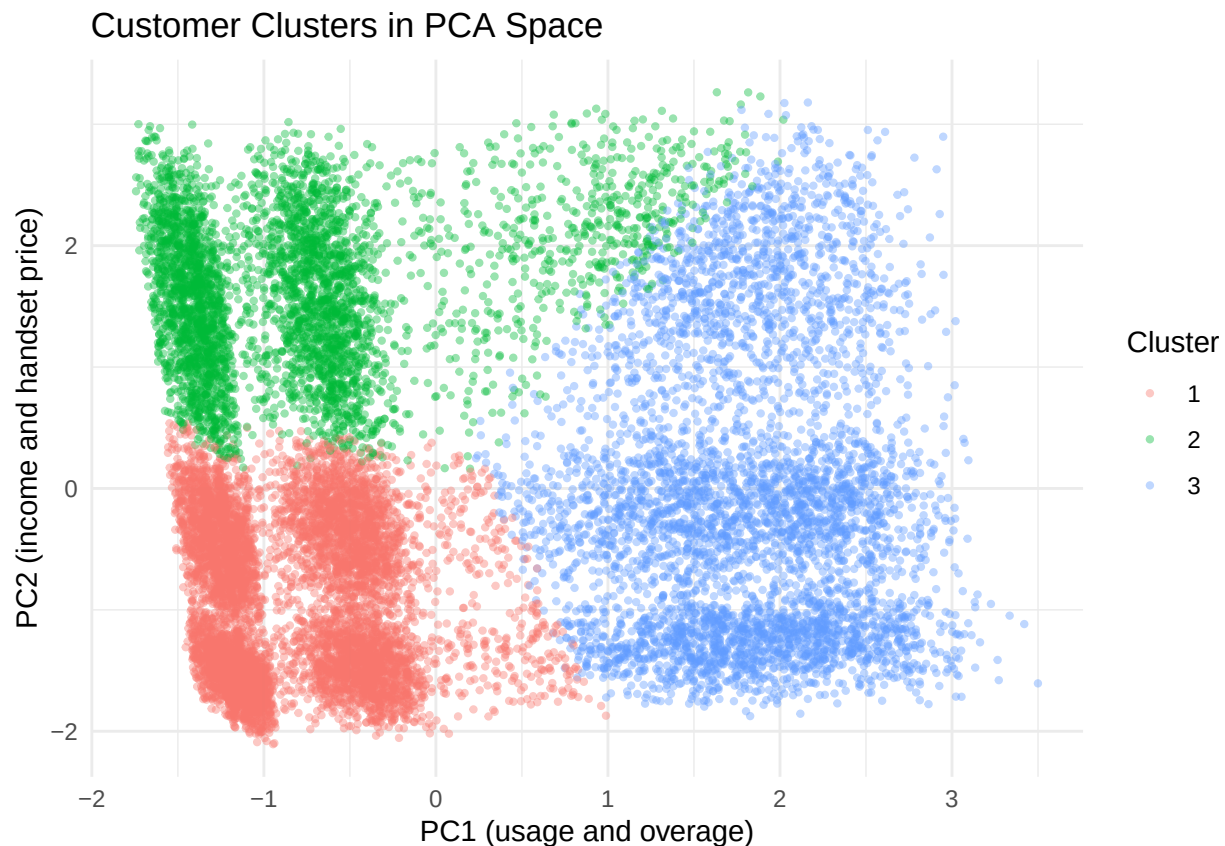
```
# Only taking PC1 and PC2 for visualisation
cluster_data <- data.frame(PCA1 = pca$x[, 1], PCA2 = pca$x[, 2],
                           Cluster = as.factor(kmeans_cluster_model1$cluster))

# Adding cluster labels to the original (unscaled) dataset for profiling
TrainData4_original$Cluster <- kmeans_cluster_model1$cluster

# Cluster sizes
table(TrainData4_original$Cluster)
```

```
##
##      1      2      3
## 7099 4068 4831

# plotting the clusters in PCA space
ggplot(cluster_data, aes(x = PCA1, y = PCA2, color = Cluster)) +
  geom_point(alpha = 0.4, size = 0.8) +
  labs(title = "Customer Clusters in PCA Space",
       x = "PC1 (usage and overage)",
       y = "PC2 (income and handset price)",
       colour = "Cluster") +
  theme_minimal()
```



The chart shows the three clusters projected onto the first two principal components. The main division runs along PC1: Cluster 3 (blue, 4,831 customers) contains customers with high overage and frequent long calls, and spans the full range of incomes. Among the lower-usage customers on the left, the split is along PC2, which represents affluence. Cluster 1 (red, 7,099 customers) covers lower-income customers with cheaper handsets and is the largest segment, while Cluster 2 (green, 4,068 customers) covers higher-income customers with more expensive handsets.

The boundaries between clusters are sharp straight lines with no gaps between them, which reflects what the elbow plot indicated: k-means is dividing a continuous spread of customers rather than recovering groups that exist naturally in the data. The vertical banding on the left of the chart comes from OVER_15MINS_CALLS_PER_MONTH taking small whole-number values, so many customers share identical scores.

```
table(TrainData4_original$Cluster, TrainData4_original$LEAVE)
```

```
##
##      LEAVE STAY
##    1  2757 4342
##    2  2061 2007
##    3  3062 1769
```

```
round(prop.table(table(TrainData4_original$Cluster, TrainData4_original$LEAVE), 1) * 100, 1)
```

```
##
##      LEAVE STAY
##    1  38.8 61.2
##    2  50.7 49.3
##    3  63.4 36.6
```

Although the clustering model does not use the churn label, cross-tabulating cluster membership against LEAVE shows a clear relationship. Cluster 3, the high-coverage segment, has a churn rate of 63.4%, compared with 50.7% in Cluster 2 and 38.8% in Cluster 1, against an overall rate of 49.3%. The segmentation therefore separates customers by churn risk as a by-product of grouping them by usage, and it independently supports the decision tree's finding that overage is the strongest churn signal.

```
# visualising these clusters on the original (unscaled) data with overage and income on the axes
ggplot(TrainData4_original, aes(x = OVERAGE, y = INCOME, colour = as.factor(Cluster))) +
  geom_point(alpha = 0.4, size = 0.8) +
  labs(title = "Customer Clusters by Overage and Income",
       x = "Monthly overage (£)",
       y = "Annual income (£)",
       colour = "Cluster") +
  scale_y_continuous(labels = scales::comma) +
  theme_minimal()
```



Interpretation of the chart

Plotting the same clusters against the original variables shows the segmentation in units the business can act on. Cluster 3 (blue) consists of customers with monthly overage above roughly £130, spanning all income levels. The remaining customers, all with overage below about £100, are divided by income at roughly £100,000: Cluster 1 (red) below that level and Cluster 2 (green) above it. The vertical line at zero represents the large group of customers who never exceed their allowance.

There is a visible gap between about £100 and £130 of overage where very few customers fall, which is a genuine feature of the data and contributes to the clean separation of the high-overage segment. Some points of one colour appear within another cluster's region, because these two variables are only two of the eight used to form the clusters; customers can be separated on a dimension not shown here, such as the number of long calls per month.

Recommendation

Retention effort should be concentrated on Cluster 3. This segment accounts for 30% of the customer base (4,831 customers) and churns at 63.4%, compared with an overall rate of 49.3%. It is defined by monthly overage above roughly £130 and a high volume of long calls, and it spans all income levels, so it cannot be targeted by income alone. Reviewing the price plans of customers who consistently exceed their allowance is the most promising way to reduce churn, particularly as high overage is also the strongest split in the decision tree, meaning two very different methods point to the same driver.

Cluster 2 (4,068 customers, 50.7% churn) sits close to the average and should be kept under review rather than targeted with spend right away. Cluster 1 is the largest and most loyal segment (7,099 customers, 38.8% churn) and requires minimal retention spend, freeing budget for the higher-risk groups.

There are two caveats. The clusters are imposed on a continuous spread of customers rather than recovered from naturally separated groups, so the boundaries should be treated as a practical segmentation rather than as fixed customer types. The relationship between overage and churn is also associative, so a price plan review should be tested on a sample before being rolled out.

Reference List

Dreiseitl, S. and Ohno-Machado, L., 2002. *Logistic regression and artificial neural network classification models: a methodology review.* *Journal of Biomedical Informatics*, 35(5-6), pp.352-359.

Hatem, G., Zeidan, J., Goossens, M. and Moreira, C., 2022. *Normality testing methods and the importance of skewness and kurtosis in statistical analysis.* *BAU Journal - Science and Technology*, 3(2), Article 7. Available at: https://www.researchgate.net/publication/367669081_NORMALITY_TESTING_METHODS_AND_THE_IMPORTANCE_OF_SKEWNESS_AND_KURTOSIS_IN_STATISTICAL_ANALYSIS [Accessed 20 January 2025].

Ozdemir, O., 2016. *A comparison study of data transformation methods to achieve normality.* *International Journal of Mathematical and Computational Methods*, [online] 1(1), pp.16-22. Available at: <https://iaras.org/iaras/filedownloads/ijmcm/2016/001-0055.pdf> [Accessed 20 January 2025].

The R Documentation, n.d. *trainControl function - caret package.* [online] Available at: <https://www.rdocumentation.org/packages/caret/versions/6.0-92/topics/trainControl> [Accessed 20 January 2025].

Appendix

This section records the other decision-tree and logistic regression models built while selecting the final models for Tasks 1 and 2, so that the selection can be checked. The models were fitted on different train/test splits, so their accuracies are not directly comparable to one another.

Data partition

```
set.seed(100)
train_index <- sample(seq_len(nrow(customers)), size = nrow(customers)*0.7,replace=FALSE)
TrainData <- customers[train_index,]
TestData <- customers[-train_index,]

set.seed(100)
train_index2 <- sample(seq_len(nrow(customers)), size = nrow(customers)*0.8,replace=FALSE)
TrainData2 <- customers[train_index2,]
TestData2 <- customers[-train_index2,]

set.seed(100)
train_index3 <- createDataPartition(y=customers$LEAVE,p=0.7,list=FALSE)
TrainData3 <- customers[train_index3,]
TestData3 <- customers[-train_index3,]

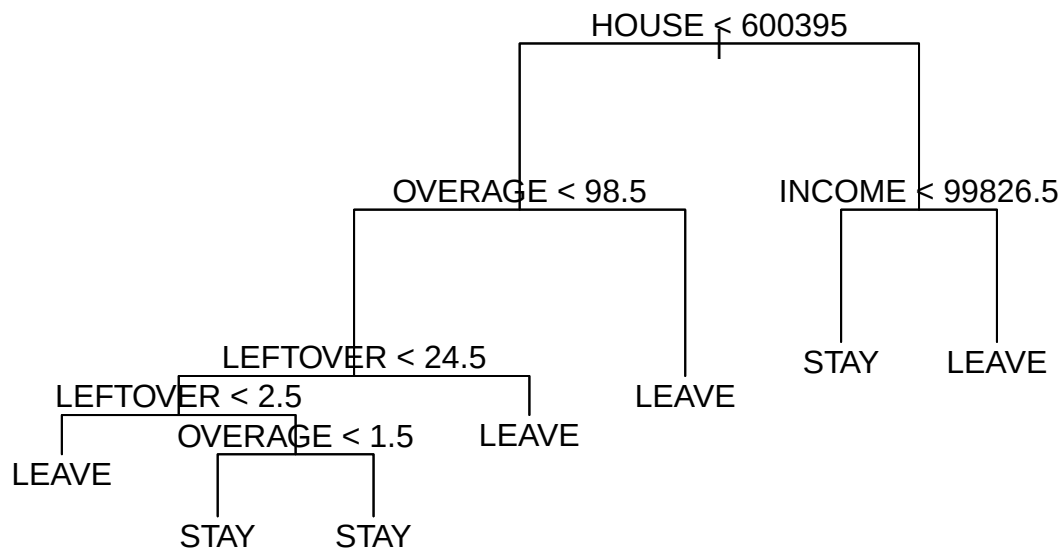
set.seed(100)
train_index4 <- createDataPartition(y=customers$LEAVE,p=0.8,list=FALSE)
TrainData4 <- customers[train_index4,]
TestData4 <- customers[-train_index4,]
```

Decision-Tree Model 1 TrainData (70:30 ratio) - tree package

```
set.seed(100)
Tree1 <- tree(formula = LEAVE ~ .,
              data = TrainData)
summary(Tree1)

##
## Classification tree:
## tree(formula = LEAVE ~ ., data = TrainData)
## Variables actually used in tree construction:
## [1] "HOUSE" "OVERAGE" "LEFTOVER" "INCOME"
## Number of terminal nodes: 7
## Residual mean deviance: 1.155 = 16160 / 13990
## Misclassification error rate: 0.3095 = 4332 / 13997
```

```
#Plotting the tree
plot(Tree1, "A Basic Decision Tree")
text(Tree1)
```



checking Insample accuracy

```
Tree1Pred <- predict(object = Tree1,
                    newdata = TrainData,
                    type = "class")
```

```
mean(Tree1Pred == TrainData$LEAVE)
```

```
## [1] 0.6905051
```

```
#confusionMatrix(Tree1Pred, TrainData$LEAVE)
```

checking Out-sample accuracy

```
Tree1Pred <- predict(object = Tree1,  
                     newdata = TestData,  
                     type = "class")
```

```
mean(Tree1Pred == TestData$LEAVE)
```

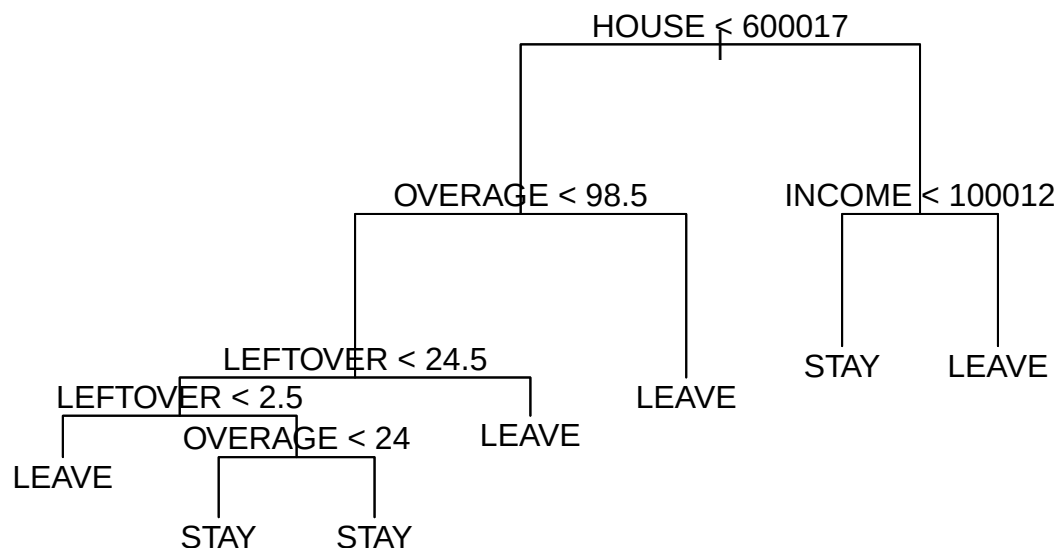
```
## [1] 0.7016169
```

Decision-Tree Model 2 TrainData2 (80:20 ratio) - tree package

```
set.seed(100)  
Tree2 <- tree(formula = LEAVE ~ .,  
              data = TrainData2)  
summary(Tree2)
```

```
##  
## Classification tree:  
## tree(formula = LEAVE ~ ., data = TrainData2)  
## Variables actually used in tree construction:  
## [1] "HOUSE" "OVERAGE" "LEFTOVER" "INCOME"  
## Number of terminal nodes: 7  
## Residual mean deviance: 1.156 = 18490 / 15990  
## Misclassification error rate: 0.3092 = 4946 / 15996
```

```
#Plotting the tree  
plot(Tree2, "A Basic Decision Tree")  
text(Tree2)
```



checking Insample accuracy

```
Tree2Pred <- predict(object = Tree2,
                     newdata = TrainData2,
                     type = "class")

mean(Tree2Pred == TrainData2$LEAVE)
```

```
## [1] 0.6907977
```

checking Out-sample accuracy

```
Tree2Pred <- predict(object = Tree2,
                     newdata = TestData2,
                     type = "class")

mean(Tree2Pred == TestData2$LEAVE)
```

```
## [1] 0.7065
```

Decision-Tree Model 3 TrainData (70:30 ratio) - tree package with modifications

```

set.seed(100)
Tree3 <- tree(formula = LEAVE ~ .,
               data = TrainData, control= tree.control(nobs= nrow(TrainData), mincut = 0, minsize = 1
summary(Tree3)

```

```

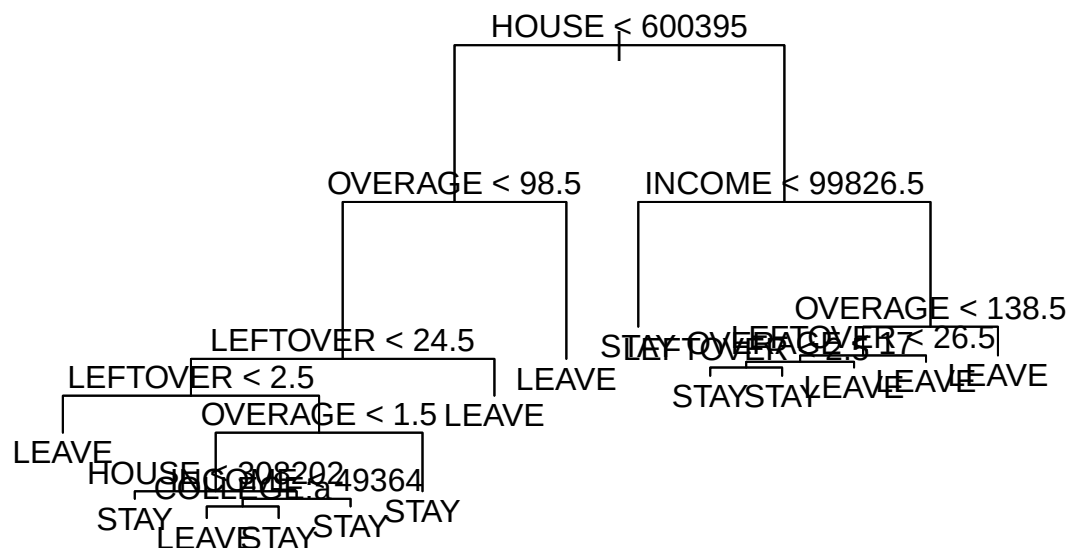
##
## Classification tree:
## tree(formula = LEAVE ~ ., data = TrainData, control = tree.control(nobs = nrow(TrainData),
##   mincut = 0, minsize = 1, mindev = 0.001))
## Variables actually used in tree construction:
## [1] "HOUSE"    "OVERAGE"  "LEFTOVER" "INCOME"   "COLLEGE"
## Number of terminal nodes:  14
## Residual mean deviance:  1.125 = 15730 / 13980
## Misclassification error rate: 0.2981 = 4172 / 13997

```

#Plotting the tree

```
plot(Tree3)
```

```
text(Tree3)
```



The tree does not produce a desirable plot although the accuracy has increased
checking Insample accuracy

```
Tree3Pred <- predict(object = Tree3,  
                     newdata = TrainData,  
                     type = "class")
```

```
mean(Tree3Pred == TrainData$LEAVE)
```

```
## [1] 0.7019361
```

checking Out-sample accuracy

```
Tree3Pred <- predict(object = Tree3,  
                     newdata = TestData,  
                     type = "class")
```

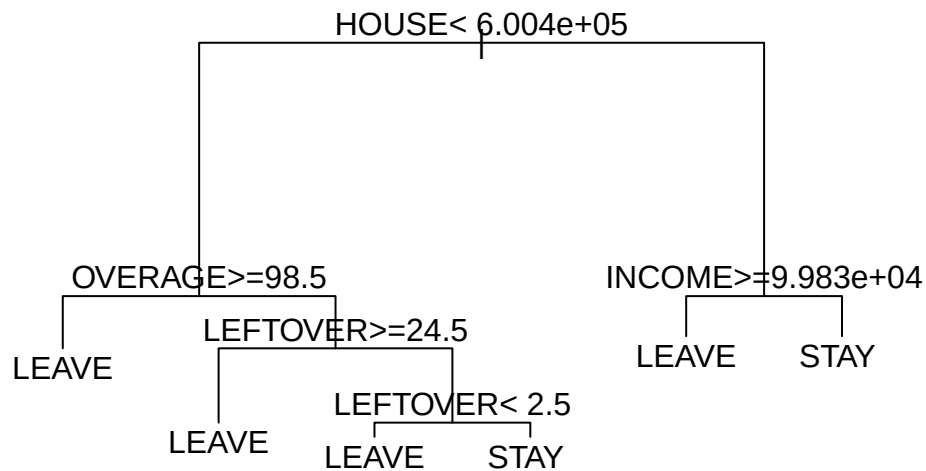
```
mean(Tree3Pred == TestData$LEAVE)
```

```
## [1] 0.7122854
```

Decision-Tree Model 4 TrainData (70:30 ratio) rpart package

```
set.seed(100)  
Tree4 <- rpart(formula = LEAVE ~ .,  
               data = TrainData)
```

```
#Plotting the tree  
plot(Tree4,margin = .1)  
text(Tree4)
```



checking Insample accuracy

```
Tree4Pred <- predict(object = Tree4,
                     newdata = TrainData,
                     type = "class")

mean(Tree4Pred == TrainData$LEAVE)
```

```
## [1] 0.6905051
```

checking Out-sample accuracy

```
Tree4Pred <- predict(object = Tree4,
                     newdata = TestData,
                     type = "class")

mean(Tree4Pred == TestData$LEAVE)
```

```
## [1] 0.7016169
```

Decision-Tree Model 5 TrainData3 (70:30 ratio) rpart package with train function with gini index

```
set.seed(100)
Tree5 <- train(LEAVE~., data = TrainData3, method = 'rpart')
Tree5Pred <- predict(Tree5, newdata=TrainData3)
confusionMatrix(Tree5Pred, TrainData3$LEAVE)
```

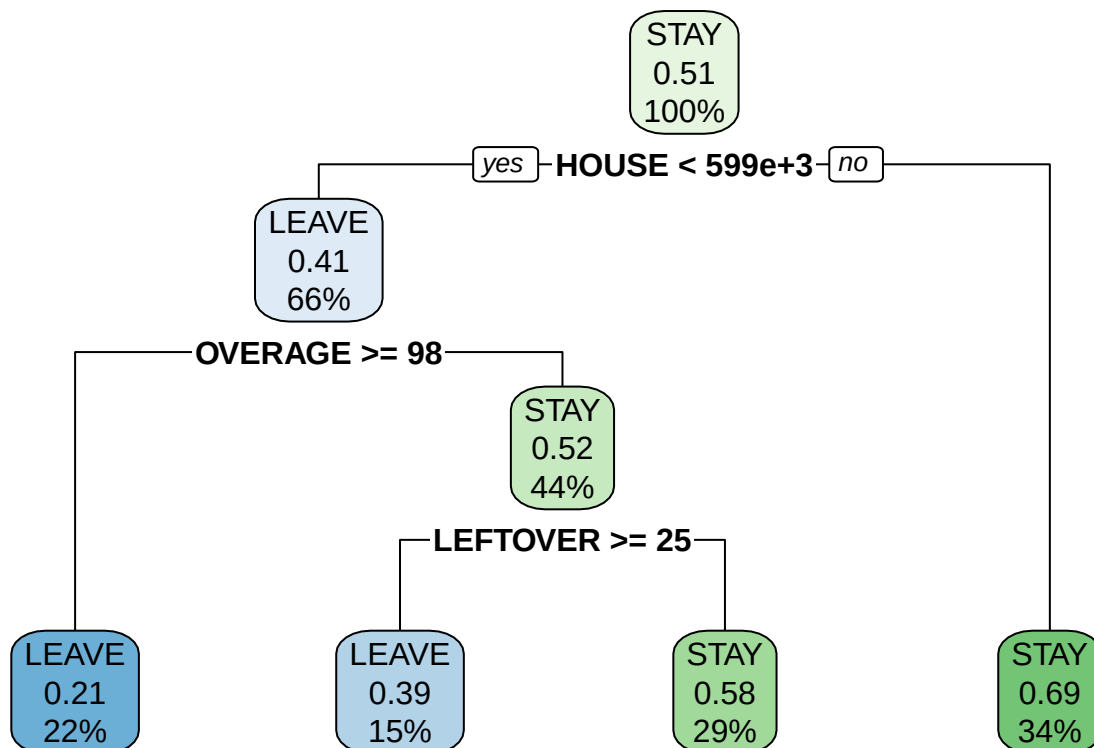
```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction LEAVE STAY
##      LEAVE   3703 1437
##      STAY    3192 5666
##
##              Accuracy : 0.6693
##              95% CI : (0.6614, 0.6771)
##      No Information Rate : 0.5074
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.336
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##      Sensitivity : 0.5371
##      Specificity : 0.7977
##      Pos Pred Value : 0.7204
##      Neg Pred Value : 0.6396
##      Prevalence : 0.4926
##      Detection Rate : 0.2645
##      Detection Prevalence : 0.3672
##      Balanced Accuracy : 0.6674
##
##      'Positive' Class : LEAVE
##
```

```
Tree5Pred <- predict(Tree5, newdata=TestData3)
confusionMatrix(Tree5Pred, TestData3$LEAVE)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction LEAVE STAY
##      LEAVE   1586  618
##      STAY    1368 2426
##
##              Accuracy : 0.6689
##              95% CI : (0.6568, 0.6808)
##      No Information Rate : 0.5075
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.3351
```

```
##
## McNemar's Test P-Value : < 2.2e-16
##
##      Sensitivity : 0.5369
##      Specificity : 0.7970
##      Pos Pred Value : 0.7196
##      Neg Pred Value : 0.6394
##      Prevalence : 0.4925
##      Detection Rate : 0.2644
##      Detection Prevalence : 0.3675
##      Balanced Accuracy : 0.6669
##
##      'Positive' Class : LEAVE
##
```

```
rpart.plot(Tree5$finalModel)
```



Decision-Tree Model 6 TrainData3 (70:30 ratio) rpart package with train function and Information Gain

```
set.seed(100)
Tree6 <- train(LEAVE~., data = TrainData3, method = 'rpart', parms = list(split="information"))
```

```
Tree6Pred <- predict(Tree6,newdata=TrainData3)
confusionMatrix(Tree6Pred, TrainData3$LEAVE)
```

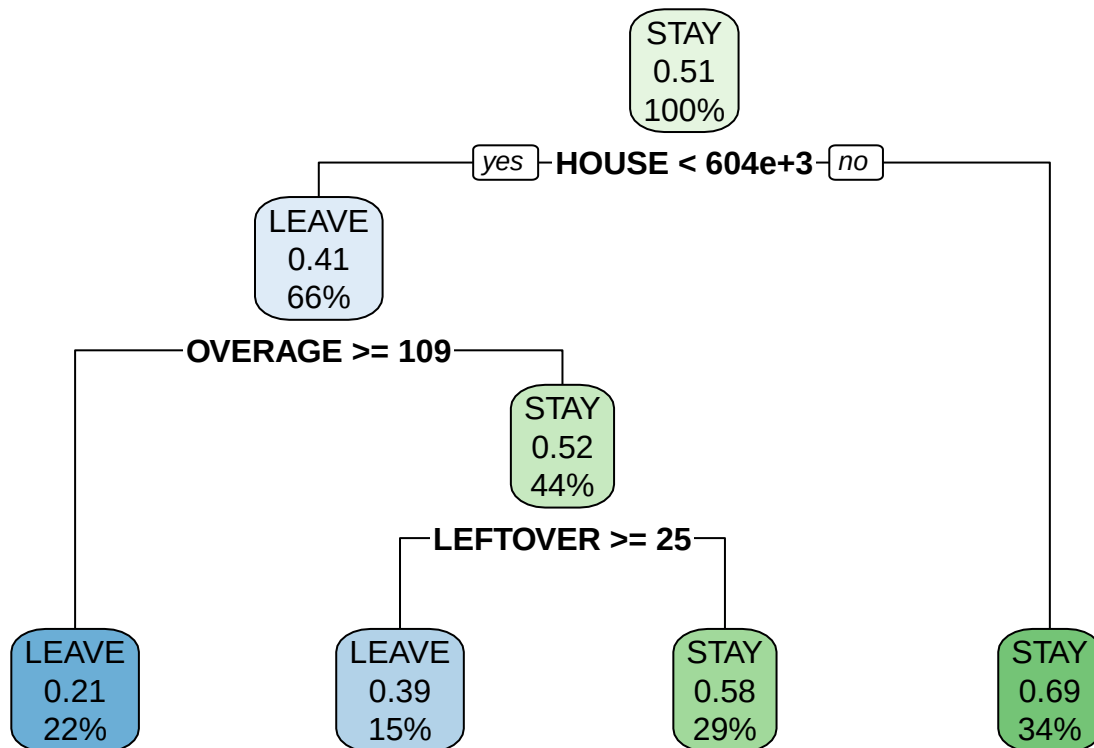
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction LEAVE STAY
##      LEAVE  3714 1445
##      STAY   3181 5658
##
##              Accuracy : 0.6695
##              95% CI : (0.6617, 0.6773)
##      No Information Rate : 0.5074
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.3365
##
##  McNemar's Test P-Value : < 2.2e-16
##
##      Sensitivity : 0.5387
##      Specificity : 0.7966
##      Pos Pred Value : 0.7199
##      Neg Pred Value : 0.6401
##      Prevalence : 0.4926
##      Detection Rate : 0.2653
##      Detection Prevalence : 0.3686
##      Balanced Accuracy : 0.6676
##
##      'Positive' Class : LEAVE
##
```

```
Tree6Pred <- predict(Tree6,newdata=TestData3)
confusionMatrix(Tree6Pred, TestData3$LEAVE)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction LEAVE STAY
##      LEAVE  1598  624
##      STAY   1356 2420
##
##              Accuracy : 0.6699
##              95% CI : (0.6578, 0.6818)
##      No Information Rate : 0.5075
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.3372
##
##  McNemar's Test P-Value : < 2.2e-16
##
##      Sensitivity : 0.5410
##      Specificity : 0.7950
##      Pos Pred Value : 0.7192
```

```
##          Neg Pred Value : 0.6409
##          Prevalence : 0.4925
##          Detection Rate : 0.2664
##          Detection Prevalence : 0.3705
##          Balanced Accuracy : 0.6680
##
##          'Positive' Class : LEAVE
##
```

```
rpart.plot(Tree6$finalModel)
```



Decision-Tree Model 7 TrainData3 (70:30 ratio) create data partition and rpart function

```
set.seed(100)
Tree7 <- rpart(LEAVE ~ ., data = TrainData3, method = "class")
Tree7Pred <- predict(Tree7, TrainData3, type = "class")
confusionMatrix(Tree7Pred, TrainData3$LEAVE)
```

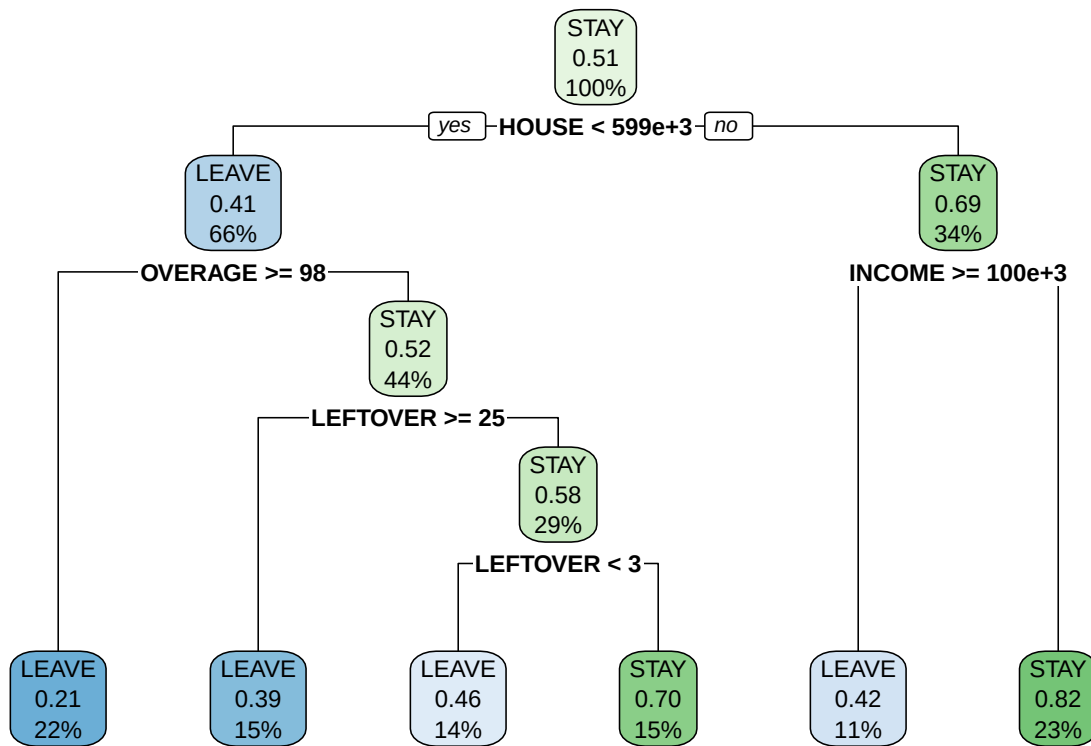
```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction LEAVE STAY
```

```
##      LEAVE  5707 3039
##      STAY   1188 4064
##
##              Accuracy : 0.698
##              95% CI : (0.6903, 0.7056)
##      No Information Rate : 0.5074
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.3983
##
##      McNemar's Test P-Value : < 2.2e-16
##
##              Sensitivity : 0.8277
##              Specificity : 0.5722
##      Pos Pred Value : 0.6525
##      Neg Pred Value : 0.7738
##      Prevalence : 0.4926
##      Detection Rate : 0.4077
##      Detection Prevalence : 0.6248
##      Balanced Accuracy : 0.6999
##
##      'Positive' Class : LEAVE
##
```

```
Tree7Pred <- predict(Tree7, TestData3, type = "class")
confusionMatrix(Tree7Pred, TestData3$LEAVE)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction LEAVE STAY
##      LEAVE  2416 1359
##      STAY   538 1685
##
##              Accuracy : 0.6837
##              95% CI : (0.6718, 0.6955)
##      No Information Rate : 0.5075
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.3699
##
##      McNemar's Test P-Value : < 2.2e-16
##
##              Sensitivity : 0.8179
##              Specificity : 0.5535
##      Pos Pred Value : 0.6400
##      Neg Pred Value : 0.7580
##      Prevalence : 0.4925
##      Detection Rate : 0.4028
##      Detection Prevalence : 0.6294
##      Balanced Accuracy : 0.6857
##
##      'Positive' Class : LEAVE
##
```

```
rpart.plot(Tree7)
```



other logistic regression models

```
customers_log <- glm(LEAVE ~ INCOME + OVERAGE + LEFTOVER + HOUSE + HANDSET_PRICE + AVERAGE_CALL_DURATION, data = TrainData2)
summary(customers_log)
```

```
##
## Call:
## glm(formula = LEAVE ~ INCOME + OVERAGE + LEFTOVER + HOUSE + HANDSET_PRICE +
##       AVERAGE_CALL_DURATION, family = "binomial", data = TrainData2)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   4.442e-01  6.994e-02   6.352 2.12e-10 ***
## INCOME        -3.928e-06  5.888e-07  -6.671 2.53e-11 ***
## OVERAGE       -6.037e-03  2.013e-04 -29.994 < 2e-16 ***
## LEFTOVER      -7.867e-03  8.394e-04  -9.372 < 2e-16 ***
## HOUSE         1.836e-06  6.837e-08  26.849 < 2e-16 ***
## HANDSET_PRICE -3.150e-04  1.150e-04  -2.740  0.00615 **
## AVERAGE_CALL_DURATION -2.705e-02  5.118e-03  -5.285 1.26e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##  
## (Dispersion parameter for binomial family taken to be 1)  
##  
##      Null deviance: 22168  on 15995  degrees of freedom  
## Residual deviance: 20271  on 15989  degrees of freedom  
## AIC: 20285  
##  
## Number of Fisher Scoring iterations: 4
```

```
Pred2 <- predict(customers_log, TestData2, type = "response")  
predictions_logistic <- ifelse(Pred2 > 0.5, 1, 0)  
confusion_matrix_logistic <- table(predictions_logistic, TestData2$LEAVE)  
accuracy_logistic <- sum(diag(confusion_matrix_logistic)) / sum(confusion_matrix_logistic)  
print(accuracy_logistic)
```

```
## [1] 0.6485
```